

Unit 4: PostgreSQL Basics & SQL Queries

A Comprehensive 9-Hour Masterclass on Relational Databases, SQL Commands, Data Types, Joins, and Indexing for Ruby on Rails Developers.

Course Agenda

- ▶ **Hour 1:** Intro & PostgreSQL Architecture
- ▶ **Hour 2:** Tools, pgAdmin, CLI & DB Creation
- ▶ **Hour 3:** PostgreSQL Data Types
- ▶ **Hour 4:** SQL Basics - DDL (Create, Alter, Drop)
- ▶ **Hour 5:** SQL Basics - DML (Insert, Update, Delete)
- ▶ **Hour 6:** SQL Basics - DQL (Select, Where, Order)
- ▶ **Hour 7:** Fetching Related Data & SQL Joins
- ▶ **Hour 8:** Database Indexing & Performance
- ▶ **Hour 9:** Rails Integration & ActiveRecord vs SQL

Hour 1: Architecture

Understanding the core structure and philosophy of PostgreSQL.

What is PostgreSQL?

An Object-Relational Database

PostgreSQL is a powerful, open-source object-relational database system (ORDBMS) with over 35 years of active development.

It has earned a strong reputation for reliability, feature robustness, and performance.

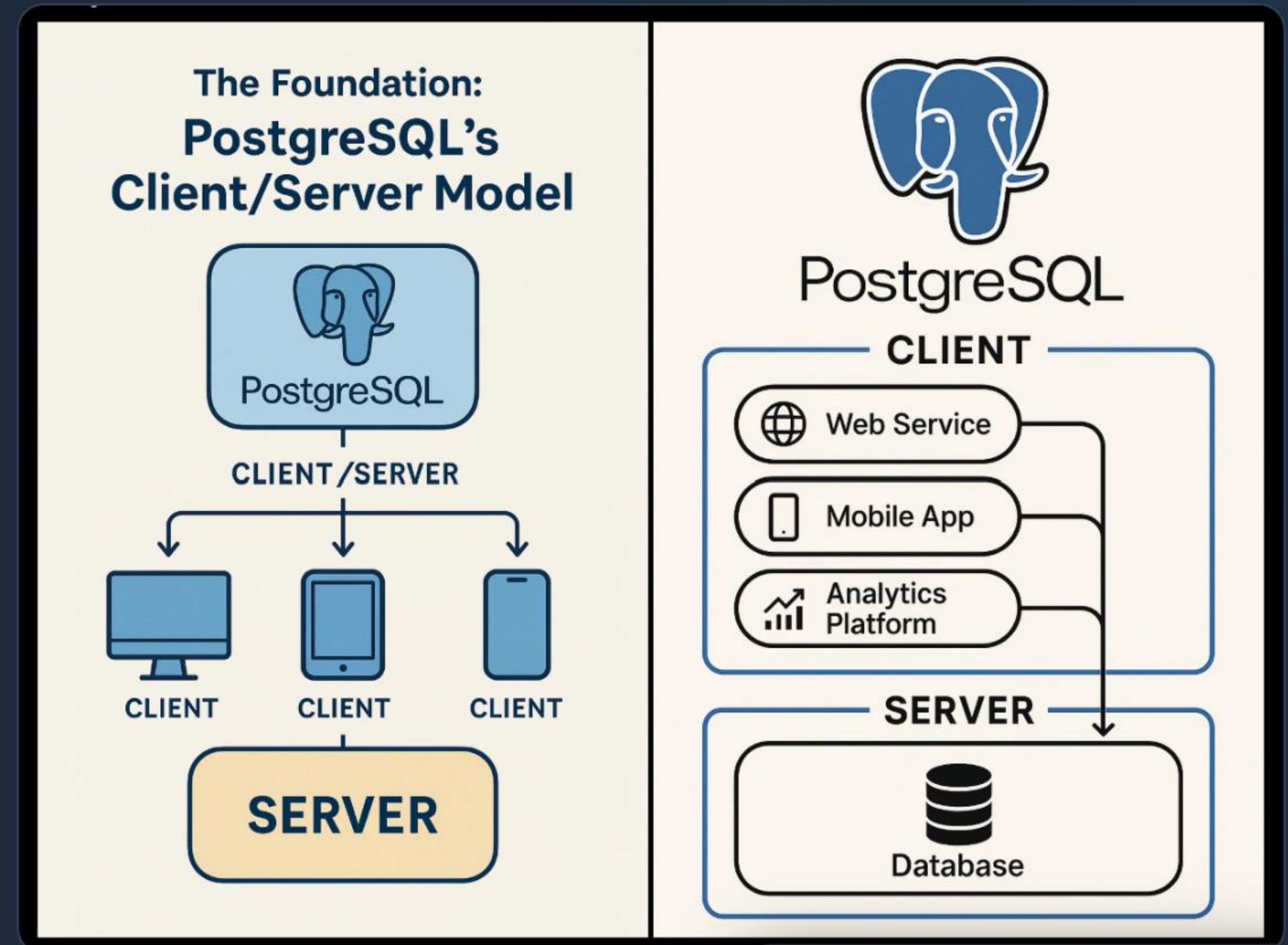
Key Differentiators

- ▶ ACID compliant (Atomicity, Consistency, Isolation, Durability)
- ▶ Supports both SQL (relational) and JSON (non-relational) querying.
- ▶ Highly extensible (custom data types, functions).
- ▶ The absolute standard database choice for Ruby on Rails.

Client-Server Architecture

PostgreSQL utilizes a classic client/server architecture model.

- ▶ **Server Process (postgres):** A background program that manages the database files, accepts connections from client applications, and performs database actions on their behalf.
- ▶ **Client Application:** The tool wanting to perform operations (e.g., your Rails app, pgAdmin, or the psql command-line tool).
- ▶ **Connection:** Clients connect via TCP/IP or local sockets. The server forks a new process for each client connection.



Why Postgres for Rails?



Native Support

Rails' ActiveRecord ORM has incredibly deep, optimized support for Postgres-specific features right out of the box.



Advanced Types

Rails can seamlessly map Ruby arrays and hashes directly to Postgres Array and JSONB column types.



Deployment

Almost all modern deployment platforms (Heroku, Render, AWS) default to PostgreSQL for Rails applications.

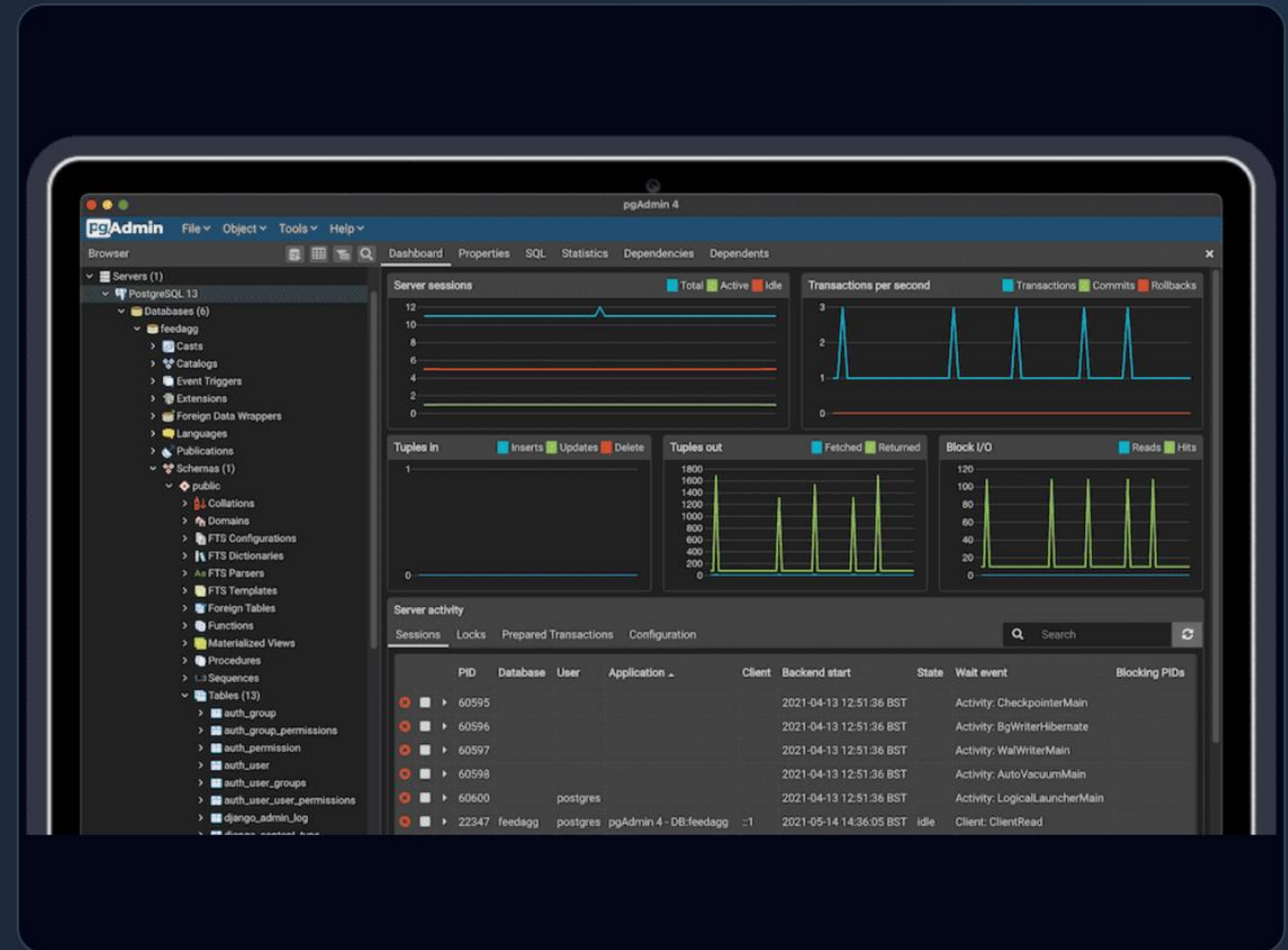
Hour 2: Tools & Creation

Interacting with the database: pgAdmin, CLI, and creating DBs.

Graphical Interface: pgAdmin

pgAdmin 4 is the leading Open Source graphical management tool for PostgreSQL.

- ▶ **Visual Query Builder:** Write, test, and debug SQL queries visually.
- ▶ **Dashboard:** Monitor active sessions, transactions, and server locks in real-time.
- ▶ **Data Viewing:** Easily browse table schemas, view rows, and manually edit data without writing SQL.
- ▶ **Backup/Restore:** Trigger database dumps and restorations via a UI.



Command Line Interface: **psql**

 **What is it?** `psql` is the terminal-based front-end to PostgreSQL. It's fast, lightweight, and scriptable.

 **Connecting:** Use `psql -U username -d database_name` to connect.

 **Listing Databases:** Type `\l` inside psql to list all databases on the server.

 **Listing Tables:** Type `\dt` to list all tables in the current connected database.

 **Quitting:** Type `\q` to exit the psql interface.

Creating a Database

Via Raw SQL

You can create a database by logging into the default postgres database and running a standard SQL command.

```
CREATE DATABASE my_app_development; CREATE DATABASE my_app_test;
```

In SQL, commands are generally case-insensitive, but capitalizing keywords is a standard convention.

Via Rails CLI

In a Ruby on Rails application, you rarely create databases manually via SQL. Instead, you use Rake tasks.

```
$ rails db:create
```

This command reads your `database.yml` file and automatically creates both the development and test databases for you.

Hour 3: Data Types

Understanding how PostgreSQL stores different formats of data.

String Data Types

PostgreSQL Type	Rails Migration Equivalent	Description & Use Case
<code>VARCHAR(n)</code>	<code>t.string</code>	Variable-length character string with a limit. Best for emails, names, and titles.
<code>TEXT</code>	<code>t.text</code>	Variable-length character string of unlimited length. Best for blog bodies, descriptions, or comments.
<code>CHAR(n)</code>	N/A (Avoid in Rails)	Fixed-length, blank-padded string. Rarely used in modern apps unless enforcing strict hardware codes.

Numeric Data Types

PostgreSQL Type	Rails Migration Equivalent	Description & Use Case
INTEGER	t.integer	Standard 4-byte whole number. Best for IDs, view counts, or standard quantities.
BIGINT	t.bigint	Large 8-byte whole number. Rails 5.1+ uses this as the default for Primary Keys (IDs).
DECIMAL(p, s)	t.decimal	Exact precision decimals. Best for Financial data/Currency where floating-point math errors are unacceptable.
FLOAT	t.float	Inexact, variable-precision numbers. Best for scientific calculations or coordinates.

Advanced Postgres **Types**



JSONB

Stores JSON data in a binary format. Allows for incredibly fast, indexed querying of schema-less document data directly inside a relational database.



Arrays

PostgreSQL allows columns of an array type (e.g., an array of strings). Rails maps this directly to a Ruby array, useful for storing a list of tags.



UUID

Universally Unique Identifiers. Often used instead of auto-incrementing integers for Primary Keys to improve security and facilitate distributed systems.

Hour 4: SQL Basics (DDL)

Data Definition Language: Defining the structure of your database.

What is DDL?



Data Definition Language (DDL) is a subset of SQL used to define or alter the database structure (the schema).



DDL commands do **not** manipulate the data inside the tables; they only manipulate the "containers" that hold the data.



Primary Commands: `CREATE`, `ALTER`, `DROP`, and `TRUNCATE`.



In Rails, you rarely write DDL SQL by hand. You use **ActiveRecord Migrations**, which generate this DDL for you.

The **CREATE TABLE** Statement

Raw SQL Syntax

Used to generate a new table. You must specify the column names and their data types.

```
CREATE TABLE users ( id SERIAL PRIMARY KEY, first_name VARCHAR(50) NOT NULL, email VARCHAR(100) UNIQUE, create
```

Constraints like **NOT NULL** and **UNIQUE** enforce data integrity at the database level.

The **ALTER TABLE** Statement

Modifying Existing Structures

Used to add, delete, or modify columns in an existing table.

```
-- Add a new column ALTER TABLE users ADD COLUMN age INTEGER; -- Change a column type ALTER TABLE users ALTER
```

Hour 5: SQL Basics (DML)

Data Manipulation Language: Creating, Updating, and Deleting records.

What is DML?



Data Manipulation Language (DML) is used for managing data within schema objects.



These are the commands you use to actually populate your database with user information, blog posts, etc.



Primary Commands: `INSERT`, `UPDATE`, and `DELETE`.



Danger: Running an UPDATE or DELETE without a WHERE clause will affect every single row in the table!

The **INSERT** Statement

Inserting a single row

Specify the table name, the columns you are targeting, and the values to insert.

```
INSERT INTO users (first_name, email) VALUES ('John', 'john@example.com');
```

Inserting multiple rows

```
INSERT INTO users (first_name, email) VALUES ('Jane', 'jane@example.com'), ('Bob', 'bob@example.com');
```

The **UPDATE** Statement

Modifying Data

Used to modify existing records. **Always use a WHERE clause** to specify which record(s) should be updated.

```
UPDATE users SET first_name = 'Jonathan', age = 30 WHERE id = 1;
```

If you omit the `WHERE id = 1`, *every* user in the database will be renamed to Jonathan!

Rails ActiveRecord Equ

Rails handles finding the record and s

```
user = User.find(1) user
```

The **DELETE** Statement

Removing Records

Used to delete existing records from a table. Just like UPDATE, a WHERE clause is critical.

```
DELETE FROM users WHERE id = 5;
```

*To remove all data quickly, **TRUNCATE TABLE users;** is much faster than a mass **DELETE**.*

Rails ActiveRecord Equivalent

ActiveRecord provides methods to easily find and destroy records.

```
user = User.find(5) user.destroy # Or targeting multiple
```

Hour 6: SQL Basics (DQL)

Data Query Language: Fetching, filtering, sorting, and grouping data.

The **SELECT** Statement

Fetching Data

The core command for querying data. You select the columns you want from a specific table.

```
-- Select specific columns SELECT first_name, email FROM users; -- Select all columns (Not recommended for prod)
```

Filtering with **WHERE**



Purpose: Used to extract only those records that fulfill a specified condition.



Operators: `=`, `<`, `>`, `>=`, `< =`, `<>` (not equal), `LIKE` (pattern matching), `IN` (multiple values).

```
-- Find users over 18 SELECT * FROM users WHERE
```

```
# Rails Equivalent User.where("age > 18") # Rail
```

Sorting & Pagination: ORDER BY & LIMIT

Sorting Data

ORDER BY sorts the result set in ascending (ASC) or descending (DESC) order.

```
SELECT * FROM users ORDER BY created_at DESC;
```

Rails equivalent: `User.order(created_at: :desc)`

Limiting Data

LIMIT restricts the number of records returned. Used often with **OFFSET** for pagination.

```
SELECT * FROM users LIMIT 10 OFFSET 20;
```

Rails equivalent: `User.limit(10).offset(20)`

Aggregation Functions

Function	SQL Example	Rails Equivalent	Use Case
<code>COUNT()</code>	<code>SELECT COUNT(*) FROM users;</code>	<code>User.count</code>	Returns the total number of rows.
<code>SUM()</code>	<code>SELECT SUM(price) FROM orders;</code>	<code>Order.sum(:price)</code>	Adds up all values in a numeric column.
<code>AVG()</code>	<code>SELECT AVG(age) FROM users;</code>	<code>User.average(:age)</code>	Calculates the average value of a column.
<code>MAX()</code> / <code>MIN()</code>	<code>SELECT MAX(score) FROM games;</code>	<code>Game.maximum(:score)</code>	Finds the highest or lowest value.

Grouping: **GROUP BY & HAVING**

GROUP BY

Groups rows that have the same values into summary rows, often used with aggregation functions.

```
-- Count users per city SELECT city, COUNT(*) FROM users GROUP BY city;
```

HAVING

Because the **WHERE** clause can

```
-- Cities with >
```

Hour 7: Relational Data

Foreign Keys, Database Joins, and fetching associated records.

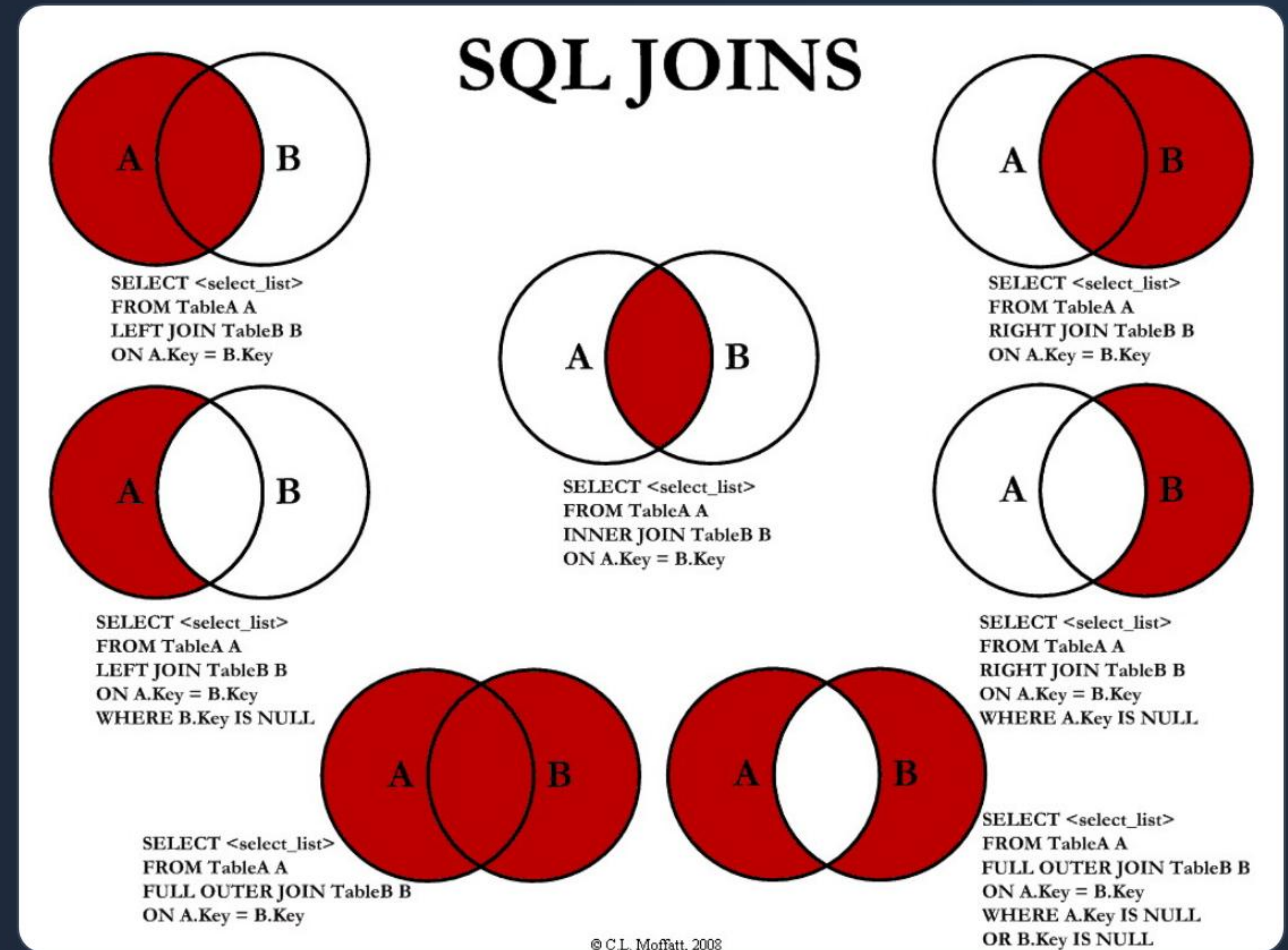
Understanding Foreign Keys

-  A **Foreign Key** is a column (or group of columns) in one table that uniquely identifies a row of another table.
-  Example: A `users` table has a Primary Key `id`. An `orders` table has a column `user_id`.
-  **Referential Integrity:** The database ensures that an order cannot be created with a `user_id` that does not exist in the `users` table.
-  In Rails, you define this via `has_many :orders` and `belongs_to :user` in your models.

Visualizing SQL Joins

When data is split across multiple tables, we use **JOIN** clauses to combine rows based on a related column between them.

- ▶ **INNER JOIN:** Returns records with matching values in both tables.
- ▶ **LEFT JOIN:** Returns all records from the left table, and matched records from the right.
- ▶ **RIGHT JOIN:** Returns all records from the right table, and matched from the left.
- ▶ **FULL OUTER JOIN:** Returns all records when there is a match in either left or right table.



The INNER JOIN

How it Works

The most common type of join. It only returns rows where there is a match in **both** tables.

```
SELECT orders.id, users.name FROM orders INNER JOIN users ON orders.user_id = users.id;
```

If a user has no orders, or an order has a null user_id, those records are excluded.

Rails Ac

Rails uses th

```
#
```

The LEFT OUTER JOIN

How it Works

Returns **all** rows from the left table (Users), and the matching rows from the right table (Orders).

```
SELECT users.name, orders.amount FROM users LEFT JOIN orders ON users.id = orders.user_id;
```

If a user has no orders, they still appear in the result, but the `orders.amount` will be `NULL`.

Rails

Rails use

Hour 8: Indexing & Perf

Making queries lightning fast using B-Trees and EXPLAIN.

The Concept of Indexing

Searching through millions of rows row-by-row (Sequential Scan) is incredibly slow.

- ▶ An **Index** is a separate data structure (usually a B-Tree) that acts like the index at the back of a book.
- ▶ Instead of reading every page to find a word, you look up the word in the index, which tells you the exact page number.
- ▶ Indexes drastically speed up **SELECT** queries, but slightly slow down **INSERT** and **UPDATE** operations (because the index must be updated too).



Creating Indexes

Raw SQL

You can create an index on any column that is frequently searched against (e.g., email lookups).

```
CREATE INDEX index_users_on_email ON users (email); -- Unique Index CREATE UNIQUE INDEX idx_uniq_email ON users (email);
```

Analyzing Queries with **EXPLAIN**

The Query Planner

PostgreSQL has a "query planner" that decides the most efficient way to execute your SQL. You can view its plan using `EXPLAIN ANALYZE`.

```
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'test@test.com';
```

It will tell you if it used a "Seq Scan" (bad for large tables) or an "Index Scan" (good).

Rails Equivalent

You can run explain directly on an A

```
> User.where(email: 'test@test.com').explain
```

Hour 9: Rails Integration

Connecting the dots between PostgreSQL and Ruby on Rails.

Configuring `database.yml`



Rails connects to PostgreSQL using the `pg` gem.



Connection settings are stored in `config/database.yml`.

```
default: &default adapter: postgresql encoding: unicode pool: <%= ENV.fet
```

ActiveRecord vs Raw SQL

The ActiveRecord Way

ORM abstracts away the SQL, preventing SQL Injection attacks and making code readable.

```
User.where(active: true) .order(created_at: :desc) .limit(5)
```

Pros: Database agnostic, secure, chainable.

Cons: Can generate inefficient queries if you aren't careful (N+1).

Falling Back to SQL

For highly complex reports or Postgres-specific queries.

```
User.find_by_sql(" SELECT * FROM users WHERE active = true ")
```

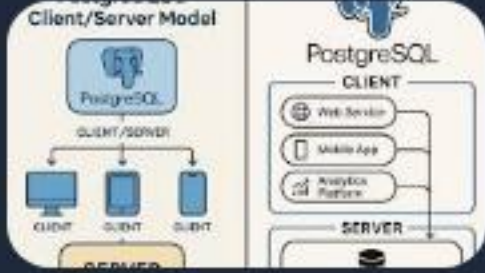
Pros: Maximum performance and flexibility.

Questions?

You have successfully completed the 9-Hour PostgreSQL Masterclass.

Keep practicing your queries!

Image Sources



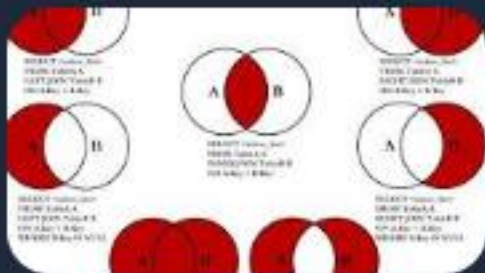
https://miro.medium.com/v2/resize:fit:1400/1*k8VeoT5phtgwKDuAJVCa_A.png

Source: medium.com



<https://www.pgadmin.org/static/COMPILED/assets/img/screenshot.png>

Source: www.pgadmin.org



<https://i.sstatic.net/UI25E.jpg>

Source: stackoverflow.com



https://www.scanning-bureau.co.uk/images/image_box/170-indexing-on-computer-conceptfw-min.png

Source: www.scanning-bureau.co.uk